

Secure Software Engineering – 20CYS401

Lab Exercise - 1

Refactoring Techniques for Enhancing Code Maintainability in Agile Projects

Learning Objectives :

- Understand the concepts of technical debt, code smells , and why code maintainability is a critical metric in Agile software projects.
- Applying clean-code refactoring techniques

You have just joined an Agile team and inherited two legacy modules. They work perfectly and pass all tests, but they suffer from high technical debt. The code is messy, hard to read, and difficult to maintain. Your objective is to pay down this technical debt using standard **Refactoring Techniques**. However, you must follow the Golden Rule of Agile Refactoring: **You must improve the structure of the code *without* changing its external behavior**. If your tests turn red, you broke it!

NOTE: Refactoring means altering the structure of code without changing its external behavior. You must run the regression test suite after every single step to ensure you haven't broken the logic. And the code base is intentionally left vulnerable; fixing the security bugs is out of this lab's scope.

Task 1

You will start with the **auth_legacy.py** module. Following the Lab Manual, you will be guided step-by-step to apply specific techniques like *Replace Magic Number*, *Guard Clauses*, and *Extract Method* to clean up the user registration flow.

```
import hashlib
```

```
def register_user(username, password, confirm_password, email, role, sec_q, sec_a, users_db):
```

```
    # Check username length (Duplicate Code)
```

```
    if len(username) < 4:
```

```
        return {"status": "REJECTED", "reason": "Username invalid length"}
```

```

if len(username) > 20:
    return {"status": "REJECTED", "reason": "Username invalid length"}
if username in users_db:
    return {"status": "REJECTED", "reason": "User exists"}

# Check password strength (Nested Conditionals & Complex Logic)
strength = "WEAK"
if len(password) >= 8:
    if any(c.isupper() for c in password):
        if any(c.isdigit() for c in password):
            strength = "STRONG"
        else:
            strength = "MEDIUM"

if strength == "WEAK":
    return {"status": "REJECTED", "reason": "Password too weak"}

if password != confirm_password:
    return {"status": "REJECTED", "reason": "Passwords mismatch"}

# Create hashes and save user (Magic Strings & Duplication)
salt = "s3cr3t_legacy_salt_2011"
pw_hash = hashlib.md5((password + salt).encode()).hexdigest()
ans_hash = hashlib.md5((sec_a.lower() + salt).encode()).hexdigest()

users_db[username] = {
    "email": email,
    "role": role,
    "password_hash": pw_hash,
    "sec_q": sec_q,

```

```
    "sec_a_hash": ans_hash,  
    "failed_attempts": 0  
}  
  
return {"status": "APPROVED", "reason": "Registered"}
```

```
def login_user(username, password, users_db):  
    # Check username length (Duplicate Code)  
    if len(username) < 4:  
        return {"status": "REJECTED", "reason": "Username invalid length"}  
    if len(username) > 20:  
        return {"status": "REJECTED", "reason": "Username invalid length"}  
  
    if username not in users_db:  
        return {"status": "REJECTED", "reason": "Invalid user"}  
  
    user = users_db[username]  
    if user["failed_attempts"] >= 3:  
        return {"status": "LOCKED", "reason": "Locked out"}  
  
    # Verify password hash (Magic Strings & Duplication)  
    salt = "s3cr3t_legacy_salt_2011"  
    pw_hash = hashlib.md5((password + salt).encode()).hexdigest()  
  
    if pw_hash != user["password_hash"]:  
        user["failed_attempts"] += 1  
        return {"status": "REJECTED", "reason": "Invalid password"}  
  
    user["failed_attempts"] = 0  
    return {"status": "APPROVED", "reason": "Welcome back"}
```

1: Replace Magic Numbers with Symbolic Constants

The Smell: Hardcoded numbers and strings (like 4, 20, 3, and "s3cr3t_legacy_salt_2011") are scattered through the code without explanation.

Hint: Define constants at the very top of your file (e.g., MAX_ATTEMPTS = 3). Replace the literal values in your code. Run your tests!

2: Extract Variable to Explain Complex Logic

The Smell: The password strength check uses complex inline logic like: *any(c.isupper() for c in password)*.

Hint: Store the result of that condition in a well-named boolean variable before using it.

```
has_upper = any(c.isupper() for c in password)
```

```
if has_upper:
```

```
# ... continue logic
```

3: Replace Nested Conditionals with Guard Clauses

The Smell: Look at the password strength check in *register_user*. It uses deeply nested if statements ("Arrow Code").

Hint: Create a helper function *check_password_strength(password)*. Instead of nesting, check for the failure conditions first and return "WEAK" immediately to exit early.

4: Extract Method (Fix Duplicate Code)

The Smell: The exact same 4 lines of code checking the username length exist in both *register_user* and *login_user*.

Hint: Create a helper function *is_valid_username(username)* that returns True or False.

5: Extract Method (Fix Long Method / Duplicate Hashing)

The Smell: Both functions contain the exact same `hashlib.md5(...)` logic. It clutters the main flow.

Hint: Create a helper function *generate_hash(value)* that adds the salt to the given string and returns the hashed result.

Task 2

Refer the provided reference documents and the code base to do the task

```
#Legacy filter
```

```
"""This module acts as a security middleware (a mini-WAF) that filters incoming JSON requests. It checks for common attack patterns like SQL Injection (SQLi) and Cross-Site Scripting (XSS), enforces rate limits, and validates request integrity before passing the data to the application."""
```

```
import time
```

```
def process_request(data, request_count):
```

```
    # Rate limit check: 5 requests per minute
```

```
    if request_count > 5:
```

```
        return "ERR_RATE_LIMIT", 0
```

```
    # Validate payload integrity
```

```
    if data is None:
```

```
        return "ERR_NULL", 0
```

```
# Check for injection patterns

for key in data:

    val = data[key]

    # SQLi patterns

    if "SELECT" in val or "DROP" in val or "DELETE" in val or "UPDATE" in val or
"INSERT" in val:

        return "ERR_SQLI", 0

    # XSS patterns

    if "<script>" in val or "alert(" in val or "onerror" in val:

        return "ERR_XSS", 0

    # Command injection

    if ";" in val or "&&" in val or "|" in val or "|" in val or "rm -rf" in val:

        return "ERR_CMD_INJ", 0

# temp_log_val = 999 # LEGACY LOGGING: DO NOT TOUCH

# Final processing

if len(val) > 100:

    return "ERR_LEN", 0

# Process the clean data
```

```
final_result = "SUCCESS"  
  
return final_result, request_count + 1
```

Example usage

```
req_data = {"user": "admin", "query": "SELECT * FROM users"}  
  
status, count = process_request(req_data, 1)  
  
print(f"Result: {status}")
```

- Read **filter_legacy.py**.
- Identify possible code smells in the code and provide the code snippet where each occurs, along with an explanation of why it is a smell.
- Apply possible refactoring techniques to eliminate these smells. For each: give the refactoring technique, the reason why it can be applied, and how it can be applied (pseudo code or code snippet).